HMI Manager

API Reference & Developer Guide

Project	BEV Truck Instrument Cluster
Platform	STM32H753ZI Nucleo-144
RTOS	Apache NuttX 12.8+
Display	ST7796 SPI 480x320 @ 16bpp
CAN	FDCAN1 - SocketCAN (500 kbps, Ext 29-bit)
Version	1.0.0 - February 2026
License	Apache License 2.0

Table of Contents

1. Architecture Overview
2. Repository Structure
3. Application Lifecycle
4. CAN Bus Interface
5. CAN Message Reference (DBC)
6. How to Add New CAN TX Messages
7. How to Handle New CAN RX Messages
8. Button Input System
9. LED Control System
10. Display and LVGL Widgets
11. How to Modify the LCD Layout
12. Command Interface (hmi_ctl)
13. Kconfig Options
14. Build and Flash
15. Testing
16. Memory Map
17. Known Issues and TODO

1. Architecture Overview

The HMI Manager is a daemon application running on Apache NuttX RTOS. It serves as the central coordinator for a Battery Electric Vehicle (BEV) truck instrument cluster, managing all human-machine interface functions.

1.1 Core Subsystems

Subsystem	Description	Interface
CAN Bus	SocketCAN TX/RX with DBC codec	/dev/can0 (FDCAN1)
Display	LVGL-based UI on ST7796 LCD	/dev/fb0 (SPI)
Buttons	11 hardware buttons via IRQ	/dev/buttons
LEDs	3 status LEDs (G/O/R)	/dev/userleds
CLI	Unix Domain Socket daemon	/tmp/hmi_cmd

1.2 Threading Model

The application uses three execution contexts. The main thread runs the event loop handling CAN RX, button polling, LVGL rendering, and Unix socket commands via select() with a 10ms timeout. Two dedicated pthreads handle periodic CAN TX at 100ms and 1000ms intervals. Pthreads are used instead of NuttX work queues because work queues run in a kernel thread context with a separate file descriptor table, causing EBADF errors when accessing the CAN socket.

Thread	Function	Period	Stack
Main loop	select() + LVGL + buttons	10ms tick	Default
CAN TX 100ms	can_send_loop_100ms()	100ms	2048 B
CAN TX 1000ms	can_send_loop_1000ms()	1000ms	2048 B

1.3 Data Flow

```
Button Press
|--> process_button_press()
| |--> Updates driver_commands_msg (global)
| |--> Updates g_direction + widget
|
| (100ms later, max latency)
|
v
can_send_loop_100ms() [pthread]
|--> car_can_driver_commands_pack()
|--> can_handler_send()
|--> CAN Bus TX --> Inverter

Inverter --> CAN Bus RX
|--> select() wakes main loop
|--> can_handler_read()
|--> process_can_frame()
| |--> Decode signal (cantools DBC)
| |--> Update g_speed / g_voltage / etc
| |--> widgets_set_*() (LVGL update)
| |--> led_control_on/off() (DC Link)
```

2. Repository Structure

```
apps/hmi_manager/
|-- app_hmi_manager.c Main daemon (entry point)
|-- hmi_ctl.c CLI client (Unix socket)
|-- Kconfig Build configuration
|-- Makefile Build rules
|-- Make.defs Application registration
|-- CMakeLists.txt CMake support
|
|-- can/
| |-- can_handler.c/h SocketCAN abstraction
| |-- car_can.c/h DBC codec (cantools)
| |-- can_trace.c/h CAN bus logger
|
|-- io_handler/
| |-- button_irq.c/h Button IRQ + debounce
| |-- led_control.c/h LED abstraction
| |-- io_handler.c/h GPIO utilities
|
|-- uiux/
| |-- fb_handler.c/h Framebuffer + LVGL init
| |-- widgets.c/h Dashboard UI components
| |-- aux_disp.c/h Auxiliary display
| |-- notifications.c/h Alert system
|
|-- cmd/
| |-- commands.c/h Command parser/dispatch
|
|-- config/
| |-- config_manager.c/h Runtime config
| |-- defaults.h Default values
|
|-- modules/
| |-- accel.c/h Accelerometer module
| |-- rfid.c/h RFID reader module
| |-- module_manager.c/h Module lifecycle
|
|-- logs/
|-- logger.c/h Logging subsystem
```

2.1 Key Files

File	Role	Modified By
app_hmi_manager.c	Main loop, threading, init sequence	Most changes
can/car_can.c/h	Auto-generated DBC codec	cantools only
can/can_handler.c/h	Socket lifecycle, send/read	Rarely
io_handler/button_irq.h	Button ID defines	When adding buttons
io_handler/led_control.c/h	LED abstraction layer	When adding LEDs
uiux/widgets.c/h	LVGL widget creation and update	UI changes
Kconfig	Build-time feature toggles	New features

3. Application Lifecycle

3.1 Initialization Sequence

The daemon initializes subsystems in strict order. Each step depends on the previous one succeeding. Critical failures (CAN, socket) cause immediate exit. Non-critical failures (LEDs, buttons) print a warning and continue.

Step	Function	Device	On Failure
1	fb_handler_init()	/dev/fb0	EXIT (if SCREEN_ENABLE)
2	led_control_init()	/dev/userleds	WARNING
3	can_handler_init()	/dev/can0	EXIT
4	can_trace_init()	-	Always OK
5	CAN TX defaults	-	Always OK
6	button_irq_init()	/dev/buttons	WARNING
7	init_command_socket()	/tmp/hmi_cmd	EXIT
8	widgets_init()	-	EXIT (if SCREEN_ENABLE)
9	pthread_create() x2	-	EXIT

3.2 Main Event Loop

```
    while (1) {
    // 1. Poll button (non-blocking)
    button = button_irq_get_pressed();
    if (button != BUTTON_INVALID)
    process_button_press(button);

    // 2. select() on CAN + Unix socket
    // timeout = LVGL_TICK_MS (10ms)
    ret = select(maxfd, &readfds, NULL, NULL, &tv);

    // 3. Handle CAN RX
    if (FD_ISSET(can_fd, &readfds))
    can_handler_read() -> process_can_frame();

    // 4. Handle CLI command
    if (FD_ISSET(cmd_fd, &readfds))
    accept() -> recv() -> process_command();

    // 5. LVGL tick (if SCREEN_ENABLE)
    lv_tick_inc(10);
    lv_timer_handler();
    }
```

4. CAN Bus Interface

4.1 can_handler API

The CAN handler provides a clean abstraction over NuttX SocketCAN. It manages interface lifecycle, socket creation, and frame I/O.

Function	Description	Returns
can_handler_init(ifname)	ifup + socket(PF_CAN) + bind	0 or -errno
can_handler_get_fd()	Get socket fd for select()	fd or -1
can_handler_send(id, ext, data, dlc)	Write CAN frame to bus	0 or -errno
can_handler_read(frame)	Read CAN frame (non-blocking)	0, -EAGAIN, -errno
can_handler_close()	Close socket	void

4.2 CAN Configuration

Parameter	Value
Interface	FDCAN1 as /dev/can0
Protocol	CAN 2.0B Extended (29-bit IDs)
Bitrate	500 kbps
Max payload	8 bytes (classic CAN)
Socket type	PF_CAN, SOCK_RAW, CAN_RAW

4.3 can_trace API

Function	Description
can_trace_init()	Initialize trace subsystem
can_trace_enable(bool)	Enable/disable frame logging via syslog
can_trace_rx(frame)	Log received frame
can_trace_tx(frame)	Log transmitted frame
can_trace_set_filter(id)	Filter by CAN ID (0 = log all)

5. CAN Message Reference (DBC)

All CAN messages are defined in the DBC file and auto-generated by cantools into car_can.c/h. All IDs are 29-bit extended.

5.1 TX Messages (HMI sends)

Message	ID	DLC	Cycle	Description
DriverCommands	0x0C03FE32	1	100ms	Drive control commands
HmiInfo	0x0C0B0032	4	1000ms	HMI heartbeat + timestamp

5.1.1 DriverCommands Bit Layout (1 byte)

Bit	Signal	Range	Description
0	demanded_drive_direction	0-1	0=Forward, 1=Backward
1	demanded_drive_state	0-1	0=Disable, 1=Enable
2	reset_inverter_errrors	0-1	0=No demand, 1=Active
3	dc_link_active_demand	0-1	0=No demand, 1=Active
4-7	pedal_setting	0-15	0=ECO, 1=Normal, 2=Sport

5.1.2 HmiInfo Layout (4 bytes)

Bytes	Signal	Type	Description
0-3	hmi1ms_ticks	uint32 LE	System uptime in ms

5.2 RX Messages (HMI receives)

Message	ID	DLC	Description
InverterSpeedInfo	0x0C0AFEFE	4	Motor speed (RPM + Hz)
SystemVoltages	0x0C06FE00	8	Battery + DC link voltages
InverterInfo	0x0C04FE00	6	DC link state + MFT state
DriverCommands	0x0C03FE32	1	Loopback (future: inverter echo)

5.2.1 InverterSpeedInfo Layout (4 bytes)

Bytes	Signal	Type	Scale	Unit
0-1	em_speed01_hz	int16 LE	1	0.1 Hz
2-3	em_speed_rpm	int14 LE	1	RPM

5.2.2 InverterInfo Layout (6 bytes)

Bytes	Signal	Type	Values
0-3	inverter1ms_ticks	uint32 LE	Uptime in ms
4	mft_state	uint8	0=Idle, 6=Run, 7=AnyStop, 10=FaultNow
5	dc_link_state	uint8 (3 bits)	0=Off, 1=PreCharge, 3=On, 4=Discharging

5.2.3 SystemVoltages Layout (8 bytes)

Bytes	Signal	Type	Scale	Unit
0-1	meas12_v	uint16 LE	0.001	V
2	meas5_v	uint8	0.1	V
3	(reserved)	-	-	-
4-5	meas_dc_link	uint16 LE	0.01	V
6-7	meas_batt_voltage	uint16 LE	0.01	V

6. How to Add New CAN TX Messages

This section describes the step-by-step process to add a new periodic CAN TX message to the HMI Manager.

Step 1: Update the DBC file

Add the new message definition to your .dbc file and regenerate the codec using cantools:

```
$ cantools generate_c_source car_can.dbc \
--output-directory can/
```

Step 2: Declare the global message struct

In app_hmi_manager.c, add a static global for the new message alongside the existing ones:

```
/* In Private Data section */
static struct car_can_my_new_msg_t my_new_msg;
```

Step 3: Initialize defaults in main()

```
/* After existing TX message defaults */
my_new_msg.some_signal = 0;
my_new_msg.another_signal = DEFAULT_VALUE;
```

Step 4a: Add to existing TX thread (same cycle)

If the new message uses the same cycle time as an existing thread (100ms or 1000ms), add the pack+send call inside the existing loop function:

```
static void *can_send_loop_100ms(void *arg)
{
uint8_t data[CAN_MAX_PAYLOAD];
uint8_t data2[CAN_MAX_PAYLOAD]; /* new msg */

while (true)
{
/* Existing message */
car_can_driver_commands_pack(...);
can_handler_send(...);

/* New message (same 100ms cycle) */
car_can_my_new_msg_pack(
data2, &my_new_msg,
CAR_CAN_MY_NEW_MSG_LENGTH);
can_handler_send(
CAR_CAN_MY_NEW_MSG_FRAME_ID,
CAR_CAN_MY_NEW_MSG_IS_EXTENDED,
data2,
CAR_CAN_MY_NEW_MSG_LENGTH);

nxsched_msleep(100);
}
}
```

Step 4b: Create a new TX thread (different cycle)

If the message requires a different cycle time, create a new thread function and add a pthread_create() in main():

```
static void *can_send_loop_500ms(void *arg)
{
uint8_t data[CAN_MAX_PAYLOAD];
while (true)
{
car_can_my_new_msg_pack(
data, &my_new_msg,
CAR_CAN_MY_NEW_MSG_LENGTH);
can_handler_send(
CAR_CAN_MY_NEW_MSG_FRAME_ID,
CAR_CAN_MY_NEW_MSG_IS_EXTENDED,
data,
CAR_CAN_MY_NEW_MSG_LENGTH);
nxsched_msleep(500);
}
return NULL;
}

/* In main(), after existing pthread_create: */
pthread_t tx_500ms_tid;
pthread_create(&tx_500ms_tid, &attr,
can_send_loop_500ms, NULL);
```

Important: Each new pthread costs 2KB stack. Prefer adding to existing threads when possible.

7. How to Handle New CAN RX Messages

Step 1: Update the DBC file

Same as TX: add the message to the .dbc and regenerate.

Step 2: Add handler in process_can_frame()

Add a new else-if branch in the process_can_frame() function. Follow the existing pattern:

```
    static void process_can_frame(
    struct can_frame *frame)
    {
    uint32_t id;
    int ret;

    id = frame->can_id & CAN_EFF_MASK;

    /* ... existing handlers ... */

    else if (id == CAR_CAN_MY_NEW_MSG_FRAME_ID)
    {
    struct car_can_my_new_msg_t msg;

    ret = car_can_my_new_msg_unpack(
    &msg, frame->data,
    frame->can_dlc);
    if (ret == 0)
    {
    /* Decode signal (apply scale) */
    double value =
    car_can_my_new_msg_signal_decode(
    msg.signal_name);

    /* Update global state */
    g_my_variable = (int)value;

    /* Update widget if screen enabled */
    #ifdef CONFIG_HMI_MANAGER_SCREEN_ENABLE
    widgets_set_my_value(g_my_variable);
    #endif
    }
    }
    }
```

Step 3: Add global variable (if needed)

```
    /* In Private Data section */
    static int g_my_variable = 0;
```

Step 4: Add widget update function

If the signal needs to be displayed, add a widgets_set_*() function in uiux/widgets.c and declare it in widgets.h. See Section 11 for details.

Checklist

#	Task	File
1	Add message to .dbc	car_can.dbc
2	Regenerate codec	can/car_can.c/h
3	Add else-if in process_can_frame()	app_hmi_manager.c
4	Declare global variable	app_hmi_manager.c
5	Add decode + scale logic	app_hmi_manager.c
6	Add widget function (optional)	uiux/widgets.c/h
7	Wrap widget calls with SCREEN_ENABLE	app_hmi_manager.c
8	Test with cansend from Linux	Terminal

8. Button Input System

The board supports 11 hardware buttons (BTN0-BTN10) managed through /dev/buttons with IRQ-driven detection and software debouncing.

8.1 Button Mapping

Button	Define	Action
BTN0	BUTTON_DC_LINK_TOGGLE	Toggle DC Link demand ON/OFF
BTN1	BUTTON_FORWARD	Enable Forward (requires DC Link ON)
BTN2	BUTTON_REVERSE	Enable Reverse (requires DC Link ON)
BTN3	BUTTON_NEUTRAL	Disable drive (Neutral)
BTN4	BUTTON_RESET_FAULTS	Reset inverter faults
BTN5	BUTTON_PEDAL_MODE	Cycle ECO/Normal/Sport
BTN6-10	(reserved)	Reserved for future use

8.2 Safety Interlock

BTN1 (Forward) and BTN2 (Reverse) are protected by a safety interlock. They are blocked unless the DC Link state, reported by the inverter via CAN message InverterInfo (0x0C04FE00), is DC_LINK_ON (value 3). This prevents the driver from engaging the motor before the high-voltage bus is energized.

8.3 button_irq API

Function	Description
button_irq_init(devpath)	Open device + configure IRQ
button_irq_get_pressed()	Returns button ID or BUTTON_INVALID
button_irq_clear()	Clear pressed state after handling
button_irq_close()	Close device

8.4 Adding a New Button Action

To assign an action to a reserved button (BTN6-BTN10):

```
    /* 1. Add define in io_handler/button_irq.h */
    #define BUTTON_MY_FEATURE 6

    /* 2. Add case in process_button_press() */
    case BUTTON_MY_FEATURE:
    /* Update CAN message state or trigger action */
    driver_commands_msg.some_field = new_value;
    printf("My feature activated\n");
    break;
```

9. LED Control System

Three user LEDs on the Nucleo board are controlled via /dev/userleds using NuttX userled ioctl interface.

9.1 LED Mapping

LED	Color	Define	Usage
0	Green	LED_DC_LINK_ON	DC Link active
1	Orange	LED_DC_LINK_PRECHARGE	DC Link pre-charging
2	Red	LED_RED	Available for faults

9.2 led_control API

Function	Description
led_control_init(devpath)	Open /dev/userleds, turn all off
led_control_on(led)	Turn on specified LED
led_control_off(led)	Turn off specified LED
led_control_close()	Close device

9.3 LED State Machine (DC Link)

DC Link State	Value	Green	Orange
DC_LINK_OFF	0	OFF	OFF
PRE_CHARGING	1	OFF	ON
PRE_CHARG_DONE	2	OFF	OFF
DC_LINK_ON	3	ON	OFF
DISCHARGING	4	OFF	OFF

10. Display and LVGL Widgets

10.1 Display Hardware

Parameter	Value
Controller	ST7796
Resolution	480 x 320 pixels
Color depth	16 bpp (RGB565)
Interface	SPI
NuttX driver	/dev/fb0 (framebuffer)
LVGL version	8.x

10.2 fb_handler API

Function	Description
fb_handler_init(devpath)	Open fb0, init LVGL, register display driver
fb_handler_close()	Close framebuffer

10.3 widgets API

Function	Description
widgets_init()	Create all dashboard UI elements
widgets_set_speed(int kmh)	Update speed display (km/h)
widgets_set_voltage(int dV)	Update voltage display (decivolts)
widgets_set_current(int dA)	Update current display (deciamps)
widgets_set_direction(int dir)	Update direction indicator (F/R/N)

10.4 Direction Constants

Constant	Value	Display
WIDGET_DIR_NEUTRAL	0	N
WIDGET_DIR_FORWARD	1	F
WIDGET_DIR_REVERSE	2	R

10.5 SCREEN_ENABLE Guard

All display-related code is wrapped with CONFIG_HMI_MANAGER_SCREEN_ENABLE. When disabled, the application saves approximately 77KB of flash by excluding LVGL, framebuffer, and

widget code. This allows headless operation for CAN-only testing.

11. How to Modify the LCD Layout

Step 1: Edit widgets_init() in uiux/widgets.c

All UI elements are created in widgets_init(). LVGL objects are positioned using lv_obj_set_pos() or lv_obj_align(). The screen is 480x320.

```
    /* Example: Add a temperature label */
    static lv_obj_t *temp_label;

    /* Inside widgets_init(): */
    temp_label = lv_label_create(lv_scr_act());
    lv_label_set_text(temp_label, "Temp: --C");
    lv_obj_set_pos(temp_label, 350, 250);
    lv_obj_set_style_text_font(
    temp_label, &lv_font_montserrat_20,
    LV_PART_MAIN);
    lv_obj_set_style_text_color(
    temp_label, lv_color_white(),
    LV_PART_MAIN);
```

Step 2: Create the update function

```
    /* In widgets.c */
    void widgets_set_temperature(int temp_c)
    {
    char buf[32];
    snprintf(buf, sizeof(buf), "Temp: %dC", temp_c);
    lv_label_set_text(temp_label, buf);
    }

    /* In widgets.h */
    void widgets_set_temperature(int temp_c);
```

Step 3: Call from process_can_frame()

```
    /* In process_can_frame(), new handler: */
    else if (id == CAR_CAN_INVERTER_TEMP_FRAME_ID)
    {
    struct car_can_inverter_temp_t msg;
    ret = car_can_inverter_temp_unpack(
    &msg, frame->data,
    frame->can_dlc);
    if (ret == 0)
    {
    int temp = (int)
    car_can_inverter_temp_inverter_temp_u_decode(
    msg.inverter_temp_u);
    #ifdef CONFIG_HMI_MANAGER_SCREEN_ENABLE
    widgets_set_temperature(temp);
    #endif
    }
    }
```

Layout Tips

Tip	Details
Screen origin	(0,0) is top-left
Coordinate system	X right, Y down
Font sizes	lv_font_montserrat_14/20/28/36/48
Background	Set on lv_scr_act() in widgets_init()
Alignment	Use lv_obj_align() for relative positioning
Refresh	LVGL auto-redraws on lv_timer_handler()

12. Command Interface (hmi_ctl)

The daemon exposes a Unix Domain Socket at /tmp/hmi_cmd. The hmi_ctl client sends commands over SOCK_STREAM.

12.1 Usage

```
nsh> hmi_ctl trace on # Enable CAN trace
nsh> hmi_ctl trace off # Disable CAN trace
nsh> hmi_ctl --help # Show available commands
```

12.2 Architecture

```
hmi_ctl (client) hmi_manager (daemon)
| |
|--- socket(AF_UNIX) --+
|--- connect() -----> accept()
|--- send(cmd) -----> recv(cmd)
| |-> process_command()
|--- close() |-> close(client_fd)
```

12.3 Adding New Commands

Commands are parsed in cmd/commands.c via commands_parse(argc, argv). To add a new command:

```
/* In commands.c */
if (strcmp(argv[1], "mycommand") == 0)
{
/* Handle command */
printf("MyCommand executed\n");
}
```

13. Kconfig Options

Option	Type	Default	Description
CONFIG_HMI_MANAGER	bool	n	Enable HMI Manager app
CONFIG_HMI_MANAGER_SCREEN_ENABLE	bool	y	Enable LVGL display
CONFIG_HMI_MANAGER_GEAR_RATIO	int	10	Gearbox ratio for RPM conversion
CONFIG_HMI_MANAGER_WHEEL_DIAMETER_MM	int	600	Wheel diameter in millimeters

13.1 Speed Conversion Formula

Vehicle speed is derived from motor RPM using the configured gear ratio and wheel diameter:

```
    wheel_rpm = motor_rpm / GEAR_RATIO
    circumference = PI * WHEEL_DIAM_M
    km_h = (wheel_rpm * circumference * 60) / 1000

    Example: 3000 RPM, ratio=10, diameter=600mm
    wheel_rpm = 300
    circ = 3.14159 * 0.6 = 1.885 m
    km/h = (300 * 1.885 * 60) / 1000 = 33.9
```

14. Build and Flash

14.1 Build

```
$ cd nuttx
$ make distclean
$ ./tools/configure.sh nucleo-h753zi:hmi
$ make -j$(nproc)
```

14.2 Flash

```
$ openocd -f interface/stlink.cfg \
-f target/stm32h7x.cfg \
-c "program nuttx.bin 0x08000000 verify reset exit"
```

14.3 Run

```
nsh> hmi_manager &

# Verify CAN TX from Linux:
$ candump can0
```

14.4 Headless Mode (no display)

Disable CONFIG_HMI_MANAGER_SCREEN_ENABLE in menuconfig to build without LVGL. Saves approximately 77KB flash. CAN, buttons, LEDs, and CLI still work normally.

15. Testing

15.1 Integration Test Script

An automated test script is provided at scripts/tests/hmi_test.sh. It manages the CAN interface lifecycle, sends test frames, captures NuttX serial output, and validates responses.

```
$ ./tests/hmi_test.sh /dev/ttyACM1 /dev/ttyACM0
# ^CAN serial ^NuttX serial
```

15.2 Manual CAN Test Commands

Test	Command
DC Link ON (LED green)	cansend can0 0C04FE00#000000000603
DC Link PRE_CHARGE (LED orange)	cansend can0 0C04FE00#000000000601
DC Link OFF (LEDs off)	cansend can0 0C04FE00#000000000600
Speed 3000 RPM	cansend can0 0C0AFEFE#0000B80B
Speed 1000 RPM	cansend can0 0C0AFEFE#0000E803
Speed 0 RPM	cansend can0 0C0AFEFE#00000000
Voltage 48.0V	cansend can0 0C06FE00#000000000000C012
Voltage 96.0V	cansend can0 0C06FE00#0000000000008025

15.3 Test Results

Test logs are saved to scripts/tests/results/ with timestamps. Each log contains the NuttX console output, CAN bus capture, and pass/fail summary.

16. Memory Map

Configuration	Flash	RAM
Full (display ON)	874 KB (41.7%)	150 KB (28.7%)
Headless (display OFF)	796 KB (38.0%)	~130 KB

16.1 Thread Stack Usage

Thread	Stack	Notes
Main task	Default (2048)	Event loop + LVGL
CAN TX 100ms	2048 B	pthread
CAN TX 1000ms	2048 B	pthread
Total thread overhead	4096 B	2 pthreads x 2KB

17. Known Issues and TODO

#	Issue	Severity	Status
1	reset_inverter_errrors stays latched at 1 after BTN4 press. Should be one-shot or edge-triggered depending on inverter firmware.	Medium	Pending inverter spec
2	rpm_to_kmh() has debug printf that should be removed or moved behind a trace flag.	Low	TODO
3	No direction feedback from inverter. Display updates from button press only, not from actual motor direction.	Medium	By design (DBC limitation)
4	Pedal mode (ECO/Normal/Sport) has no dedicated CAN message. Sent in bits 4-7 of DriverCommands but no visual feedback on LCD.	Low	TODO
5	CAN trace log functions exist but DEBUG printfs in daemon should be cleaned up for production.	Low	TODO
6	No fault display on LCD. InverterFaults (0x0402FF00) is defined in DBC but not handled in process_can_frame().	Medium	TODO
7	No inverter temperature display. InverterTemp (0x0401FF00) is in DBC but not handled.	Low	TODO